



Assumption University

Enterprise Application Development
Term Project 2

Somavatey Heng 6530354
Sokty Heng 6530353

2/2025

1. Introduction	3
2. Problem Statement	3
3. Proposed Solution	3
4. How it Work	3
5. Multi Model Strategy (ChatGPT + Gemini)	4
6. Architecture	5
7. Service Justification	5
8. High Availability, Scaling, and Fault Tolerance	6
9. Cost Estimation	6

1. Introduction

AgathaAI is a cloud-native Generative AI platform that provides secure, scalable, and resilient access to Large Language Models (LLMs) through a unified API gateway and conversational interface.

Architecturally comparable to platforms such as ChatGPT or Claude, AgathaAI focuses on solving the infrastructure and governance challenges associated with deploying Generative AI systems at enterprise scale.

2. Problem Statement

Deploying Generative AI systems at scale introduces several infrastructure and governance challenges:

- Chat-based inference is compute-intensive and often long-running
- Traffic patterns are bursty and unpredictable
- Synchronous inference increases timeout risk and degrades user experience
- Unrestricted usage leads to cost overruns and unfair resource allocation
- Conversation data may contain sensitive or proprietary information
- These challenges require a design that emphasizes scalability, cost control, resilience, and data security.

3. Proposed Solution

AgathaAI is an LLM-as-a-Service platform that provides:

- A scalable API gateway for chat-based inference
- Asynchronous processing of prompts and responses
- Tier-based rate limiting (Free, Pro, Enterprise)
- Secure storage of conversation history with encryption and access control
- Horizontally scalable infrastructure deployed across multiple availability zones

The system is built using Infrastructure-as-a-Service primitives including virtual networks, load balancers, compute instances, managed queues, and distributed storage.

4. How it Work

AgathaAI follows a multi-layer, asynchronous architecture optimized for enterprise-scale GenAI workloads.

Request Flow

1. User Interface: Users interact with a ChatGPT-style web interface built using Next.js and React. Prompts are submitted through a conversational UI.
2. API Gateway / Backend

All requests pass through a load-balanced backend that performs:

- GoogleOAuth authentication validation
 - Tier-based rate limiting
 - Request validation and sanitization
 - Initial prompt persistence in PostgreSQL
3. Service Orchestration
 - The backend delegates responsibilities to specialized services:
 - Auth Service – validates identity and access rights
 - User Service – verifies subscription tier and usage quotas
 - Chat Orchestrator (ChatBot Service) – manages conversation state a
 4. Inference Execution

The LLM Worker Pool:

- Consumes tasks from a message queue
 - Routes inference to the selected provider (OpenAI or Gemini)
 - Implements retries and failure handling
 - Stores responses with token usage metadata
5. Response Delivery

Once inference completes:

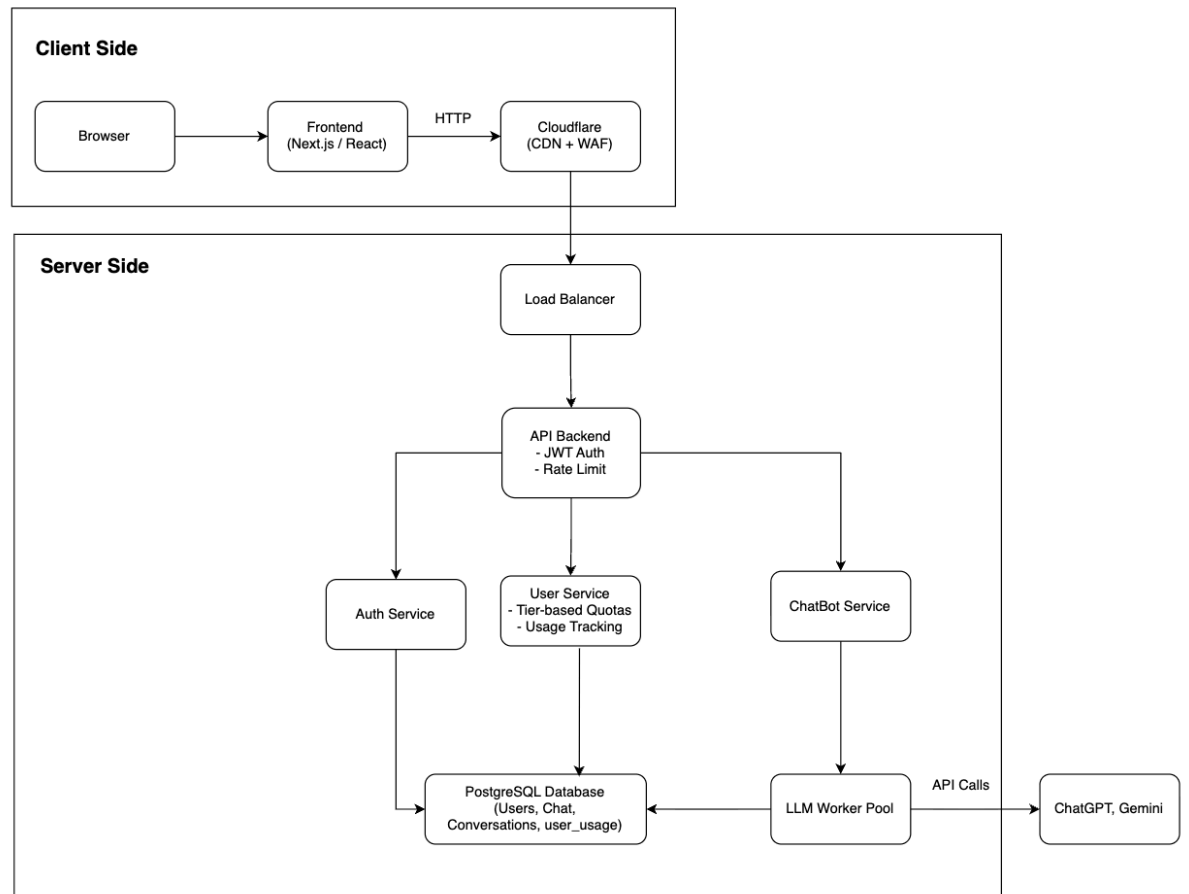
- The frontend retrieves results via polling or push notification
- Responses are streamed to the user
- Conversation history is stored with retention policies applied

This asynchronous model prevents long-running inference from blocking API request threads.

5. Multi Model Strategy (ChatGPT + Gemini)

AgathaAI employs a dual-provider LLM strategy that integrates ChatGPT (OpenAI) and Gemini (Google) behind a unified inference abstraction layer. This design addresses provider variability, cost governance, and service resilience without introducing unnecessary architectural complexity. By limiting the platform to two strategically complementary providers, AgathaAI balances operational simplicity with meaningful redundancy and cost flexibility.

6. Architecture



7. Service Justification

This section provides a formal justification for the selected infrastructure components and architectural patterns. Each design decision is mapped to the primary operational constraints of the platform: bursty traffic patterns, long-running inference workloads, cost governance, provider variability, and secure handling of sensitive conversational data.

- **DigitalOcean Managed Load Balancer:**
the centralized entry point to distribute traffic across stateless API droplets. It handles TLS termination and health checks while automatically removing unhealthy instances. This managed approach reduces operational complexity, improves availability, and ensures stable traffic distribution during unpredictable workload spikes.
- **Authentication Service (Google OAuth):**
Authentication implemented using Google OAuth is used for browser-based authentication, providing a trusted third-party identity provider and reducing the need to manage passwords directly.
- **User Service (Subscription Tiers, Quotas, and Retention Policies):**
The User Service maintains subscription tier state and associated policy parameters, including request quotas, concurrency limits, and data retention windows. By separating governance and billing-related concerns from inference execution, the

platform can evolve pricing models, usage controls, and compliance policies independently of core inference logic, improving maintainability and adaptability.

- **Chat Orchestrator / ChatBot Service:**

The Chat Orchestrator manages conversation state, constructs prompts, selects the appropriate model, and generates inference tasks. By isolating provider-specific logic, it keeps the frontend independent from LLM vendors and enables flexible routing strategies such as cost-based selection or failover. This abstraction helps manage differences in latency, cost, and availability between OpenAI GPT-4o and Google Gemini 1.5 Pro.

- **LLM Worker Pool using DigitalOcean Droplets:**

The worker pool executes inference jobs outside the synchronous request path and scales horizontally based on queue depth or resource utilization. Worker isolation limits the blast radius of failures and enables graceful degradation and provider fallback across ChatGPT, Gemini, and DeepSeek when individual providers experience outages or performance degradation.

- **Managed PostgreSQL (System of Record):**

PostgreSQL is used as the system of record for durable transactional data, including user accounts, subscription metadata, audit events, and conversation metadata. A managed high-availability deployment reduces operational complexity while providing automated failover and backups, which are essential for security-sensitive and billing-adjacent workflows.

8. High Availability, Scaling, and Fault Tolerance

The platform achieves availability and resilience through stateless service design, managed infrastructure, and asynchronous execution.

- API and worker services run as stateless replicas on DigitalOcean Droplets, with all persistent data stored in managed PostgreSQL. This allows horizontal scaling, fast instance replacement, and recovery without data loss.
- A DigitalOcean Managed Load Balancer performs continuous health checks and removes unhealthy instances automatically, ensuring stable traffic distribution and minimal manual intervention.
- Scaling is supported at both the API and worker layers. Because inference is asynchronous, queue buffering absorbs traffic spikes and prevents immediate service degradation.
- The worker pool also implements provider fallback between OpenAI GPT-4o and Google Gemini 1.5 Pro. If one provider fails or becomes slow, requests are rerouted according to predefined policies, maintaining service continuity.
- Fault tolerance is validated by simulating droplet failures and provider outages, with the system expected to remain operational despite temporary latency increases.

9. Cost Estimation

Category	Monthly Cost	Note
Load Balancer	\$12/month	DigitalOcean managed LB is HA by default (no second LB required in single region)
Droplet (2x)	\$36 (\$18x2) /month	Configure (2 vCPUs,2 GB,25 GB,3 TB) API + Worker
Database droplet	\$24/month	PostgreSQL
LLM pool	~\$300	Usage based ChatGPT and Gemini
Google Auth	\$0	Free OAuth 2.0
Domain & SSL	~\$16	Domain registration
Monitoring	\$0	Basic: DigitalOcean monitoring (free)

Estimated Cost

Currency	Amount
USD/month	\$388/month ~ \$400/month
TBH/month (~31.74TBH/USD)	~12,696 THB